

Solucionador de Sudoku

Programación Competitiva

Ramos Castillo Jefry Edinson • Ingeniería de Sistemas • Universidad Nacional del Altiplano

1. Descripción General

El proyecto implementa un **solucionador automático de Sudokus 9×9** en C++. El programa recibe un tablero parcialmente completado desde un archivo de entrada (`in.txt`), lo resuelve mediante la técnica de *backtracking recursivo*, y muestra el resultado en consola.

Lenguaje

C++ (compilado con g++)

Algoritmo

Backtracking recursivo con validación de restricciones

Archivos fuente

`sudokuOriginal.cpp` (plantilla) y
`sudokuSolucionado.cpp` (solución)

Líneas de código

~142 líneas totales (57 plantilla + 85 solución)

Repositorio del proyecto en GitHub:

<https://github.com/jefrabit/sudoku>

2. Tablero de Entrada

El archivo `in.txt` contiene 27 pistas iniciales (celdas prellenadas) en formato `fila columna valor` (1-indexado). A continuación se muestra el tablero de entrada:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

■ Pistas iniciales

□ Celdas vacías

3. Flujo del Programa

Lectura de in.txt



Parseo del tablero



Backtracking recursivo



Impresión en consola

4. Análisis de Funciones Principales

4.1 esValido() — Validador de Restricciones

Verifica si el número n puede colocarse en la posición (f, c) sin violar las reglas del Sudoku. Realiza tres comprobaciones:

- **Fila:** Recorre toda la fila f verificando que n no exista.
- **Columna:** Recorre toda la columna c verificando que n no exista.
- **Sub-cuadro 3×3:** Calcula el origen del sub-cuadro con $f - f\%3$ y $c - c\%3$, luego verifica las 9 celdas del bloque.

```
// Verifica si un numero es valido en una posicion del sudoku
bool esValido(int t[9][9], int f, int c, int n)
{
    for (int i = 0; i < 9; i++)
```

```

        if (t[f][i] == n) return false;

    for (int i = 0; i < 9; i++)
        if (t[i][c] == n) return false;

    int sf = f - f % 3;
    int sc = c - c % 3;
    for (int i = sf; i < sf + 3; i++)
        for (int j = sc; j < sc + 3; j++)
            if (t[i][j] == n) return false;

    return true;
}

```

4.2 resolver() — Backtracking Recursivo

Función central del algoritmo. Busca la primera celda vacía (valor 0) recorriendo el tablero de izquierda a derecha, arriba a abajo. Para cada celda vacía:

1. Intenta colocar los números del 1 al 9.
2. Para cada número válido, lo coloca y llama recursivamente a `resolver()`.
3. Si la recursión retorna `false`, **retrocede** (restaura la celda a 0) y prueba el siguiente número.
4. Si ninguna opción funciona, retorna `false` (backtrack completo).
5. Caso base: si no hay celdas vacías, retorna `true` (Sudoku resuelto).

```

// Resuelve el sudoku usando backtracking recursivo
bool resolver(int t[9][9])
{
    for (int f = 0; f < 9; f++)
    {
        for (int c = 0; c < 9; c++)
        {
            if (t[f][c] == 0)
            {
                for (int n = 1; n <= 9; n++)
                {
                    if (esValido(t, f, c, n))
                    {
                        t[f][c] = n;
                        if (resolver(t)) return true;
                        t[f][c] = 0;
                    }
                }
                return false;
            }
        }
    }
}

```

```
    return true;
}
```

4.3 imprimir() — Visualización del Tablero

Imprime el tablero 9×9 con separadores visuales: líneas horizontales cada 3 filas y barras verticales cada 3 columnas, facilitando la lectura del resultado.

4.4 main() — Punto de Entrada

Redirige la entrada estándar a `in.txt` mediante `freopen()`, lee las coordenadas y valores, imprime el estado inicial, resuelve el Sudoku, e imprime la solución.

5. Análisis de Complejidad

Aspecto	Complejidad	Justificación
<code>esValido()</code>	$O(27) = O(1)$	Recorre fila (9) + columna (9) + sub-cuadro (9) = 27 comparaciones constantes
<code>resolver()</code>	$O(9^k)$	Donde k = celdas vacías. En el peor caso prueba 9 opciones por cada una de las 81 celdas
Espacio	$O(k)$	Pila de recursión máxima = número de celdas vacías (máx. 81)
Caso promedio	Muy rápido	La poda de <code>esValido()</code> reduce drásticamente el árbol de búsqueda para Sudokus válidos

6. Estructura del Proyecto

Archivo	Tipo	Descripción
<code>sudokuSolucionado.cpp</code>	Código fuente	Solución completa con backtracking implementado (85 líneas)
<code>sudokuOriginal.cpp</code>	Plantilla	Esqueleto con función <code>SolucionarSudoku()</code> vacía para que el estudiante la implemente (57 líneas)
<code>in.txt</code>	Datos de entrada	Tablero Sudoku con 27 pistas iniciales (38 líneas)
<code>sudoku_ejemplo.png</code>	Imagen	Referencia visual del Sudoku resuelto (168 KB)

<code>.vscode/tasks.json</code>	Configuración	Tarea de compilación con g++ (debug symbols habilitados)
sudokuSolucionado	Binario	Ejecutable compilado (ELF 64-bit, 35 KB)

7. Conclusiones

- El proyecto demuestra la aplicación efectiva del **backtracking** para resolver problemas de satisfacción de restricciones.
- La función `esValido()` actúa como poda eficiente, eliminando ramas inválidas del árbol de búsqueda antes de explorarlas.
- La estructura de **plantilla vs. solución** indica un enfoque pedagógico: el estudiante debe implementar la lógica de resolución de forma autónoma.
- El código es conciso (~85 líneas la solución) y sigue las convenciones de programación competitiva: entrada/salida estándar, arreglos planos, y eficiencia sobre modularidad.
- No se utiliza sistema de build externo; la compilación depende exclusivamente del task runner de VS Code.